
Recurrent Neural Network (RNN) for NLP

jujbob@gmail.com

KyungTae Lim

CONTENTS

—

- ① Introduction of RNN
- ② Structure of RNN
- ③ Application of RNN

CONTENTS

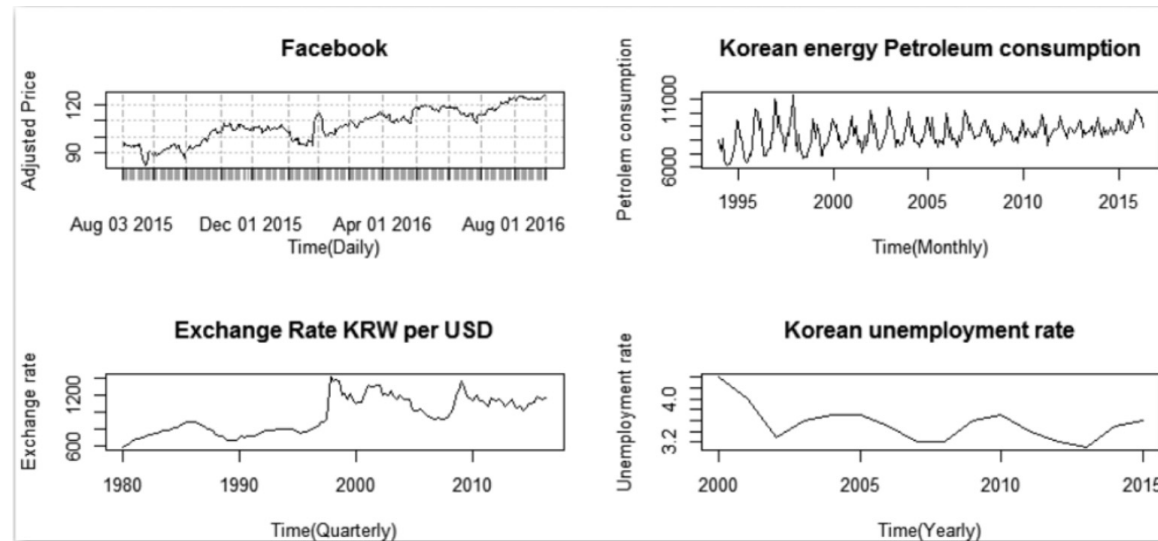
—

- ① Introduction of RNN
- ② Structure of RNN
- ③ Application of RNN

Introduction of RNN

Time Series?

- A time series is a sequence of data points arranged at consistent time intervals. (Wikipedia)
 - Time series data consists of observations ordered chronologically.
 - It is used to predict current trends and future movements based on historical data.



Introduction of RNN

Time Series?

- We aim to predict stock prices based on NCSoft's search frequency and stock price data.

How might we approach this?

- (1) Should we input all 2 years of data into an FFN? --> Too much data.
- (2) Then let's input data for five into an FFN. --> Unable to model sequential relationships
- (3) Isn't there a method to model considering previous prices over time? → RNN!!!

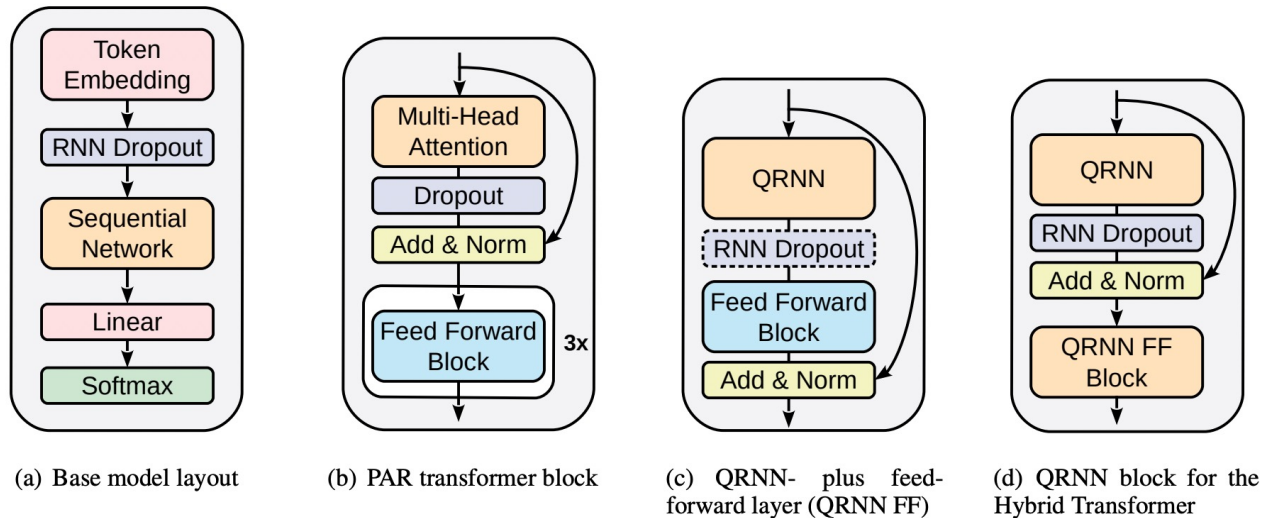


A	B	C
date	num_search	stock_price
2021.3.21	38	932000
2021.3.28	39	834000
2021.4.4	36	885000
2021.4.11	31	906000
2021.4.18	32	894000
2021.4.25	27	861000
2021.5.2	27	820000
2021.5.9	26	850000
2021.5.16	29	823000
2021.5.23	26	856000
2021.5.30	29	854000
2021.6.6	30	858000
2021.6.13	31	848000
2021.6.20	37	825000
2021.6.27	34	820000
2021.7.4	40	834000
2021.7.11	29	778000
2021.7.18	27	809000
2021.7.25	25	809000
2021.8.1	24	812000
2021.8.8	31	790000
2021.8.15	35	853000
2021.8.22	38	709000

Introduction of RNN

What is RNN(Recurrent Neural Network)?

- RNN (Recurrent Neural Network) is a deep learning model suitable for continuous and sequential data (e.g., time series data, natural language, speech).



Transformers are Multi-State RNNs

Olmo Hybrid: From Theory to Practice

William Merrill¹ Yanhong Li¹ Tyler Romero¹ Anej Svete^{1,4} Caia Costello³
Pradeep Dasigi¹ Dirk Groeneveld¹ David Heineman¹ Bailey Kuehl¹ Nathan Lambert¹
Chuan Li³ Kyle Lo^{1,2} Saumya Malik¹ DJ Matusz³ Benjamin Minixhofer^{1,5}
Jacob Morrison^{1,2} Luca Soldaini¹ Finbarr Timbers¹ Pete Walsh¹ Noah A. Smith^{1,2}
Hannaneh Hajishirzi^{1,2} Ashish Sabharwal¹

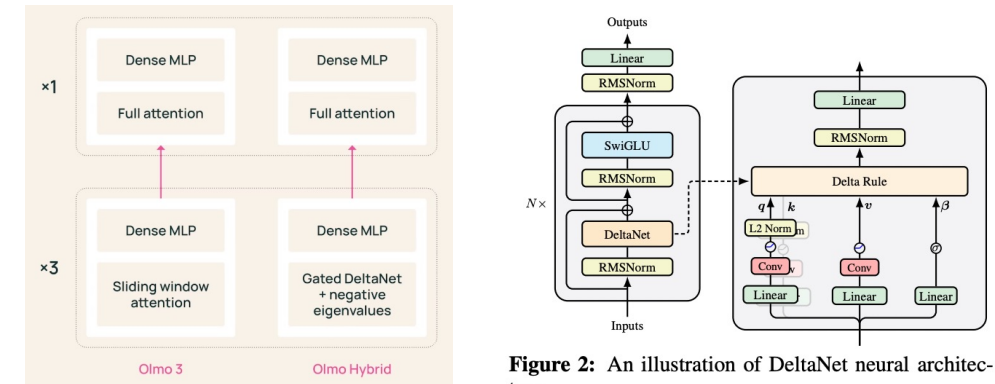


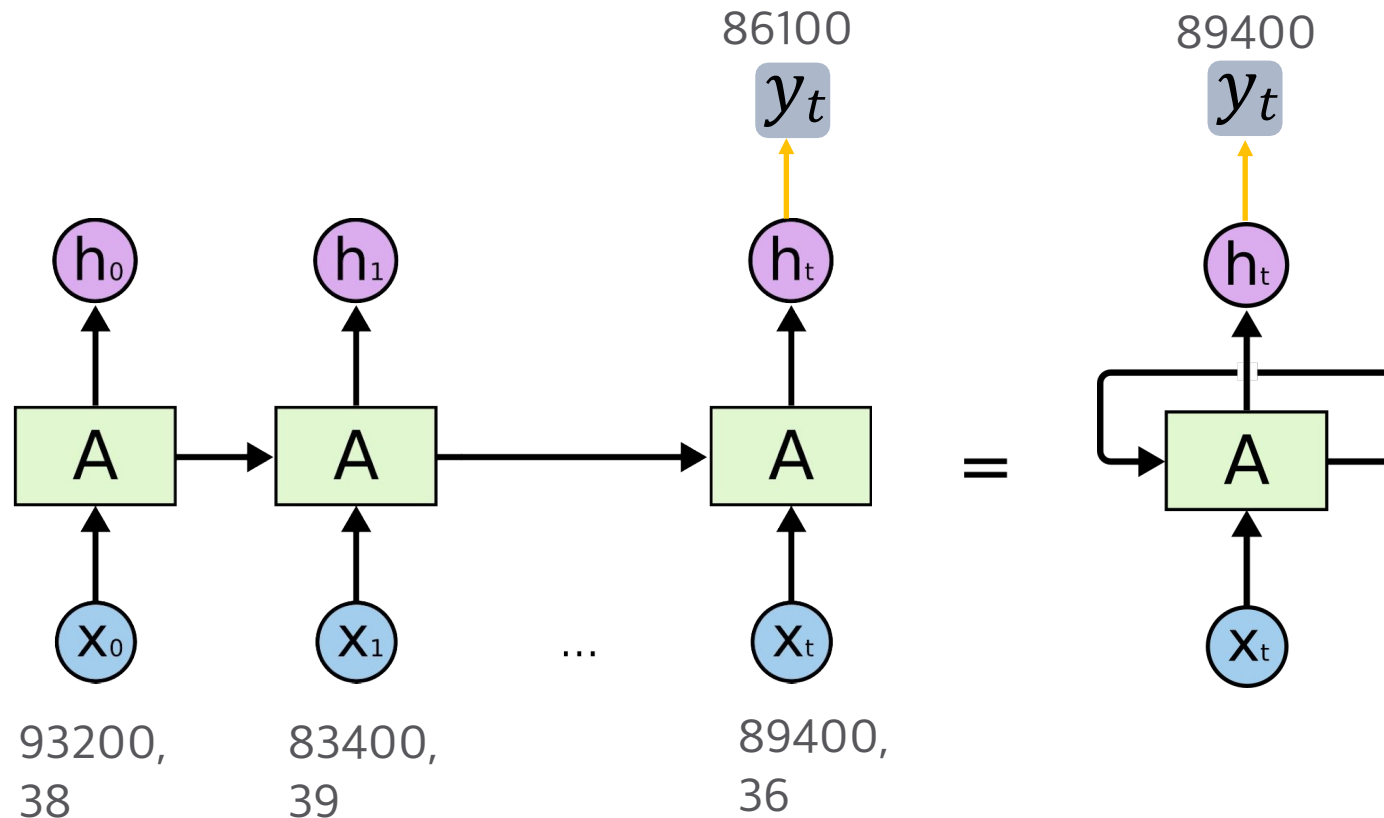
Figure 2: An illustration of DeltaNet neural architecture.

DeltaNet

Introduction of RNN

✎ What is RNN(Recurrent Neural Network)?

- RNN (Recurrent Neural Network) is a deep learning model suitable for continuous and sequential data (e.g., time series data, natural language, speech).



A	B	C
date	num_search	stock_price
2021.3.21	38	932000
2021.3.28	39	834000
2021.4.4	36	885000
2021.4.11	31	906000
2021.4.18	32	894000
2021.4.25	27	861000
2021.5.2	27	820000
2021.5.9	26	850000
2021.5.16	29	823000
2021.5.23	26	856000
2021.5.30	29	854000
2021.6.6	30	858000
2021.6.13	31	848000
2021.6.20	37	825000
2021.6.27	34	820000
2021.7.4	40	834000
2021.7.11	29	778000
2021.7.18	27	809000
2021.7.25	25	809000
2021.8.1	24	812000
2021.8.8	31	790000
2021.8.15	35	853000
2021.8.22	38	709000

Introduction of RNN

🖋 intuition of RNN

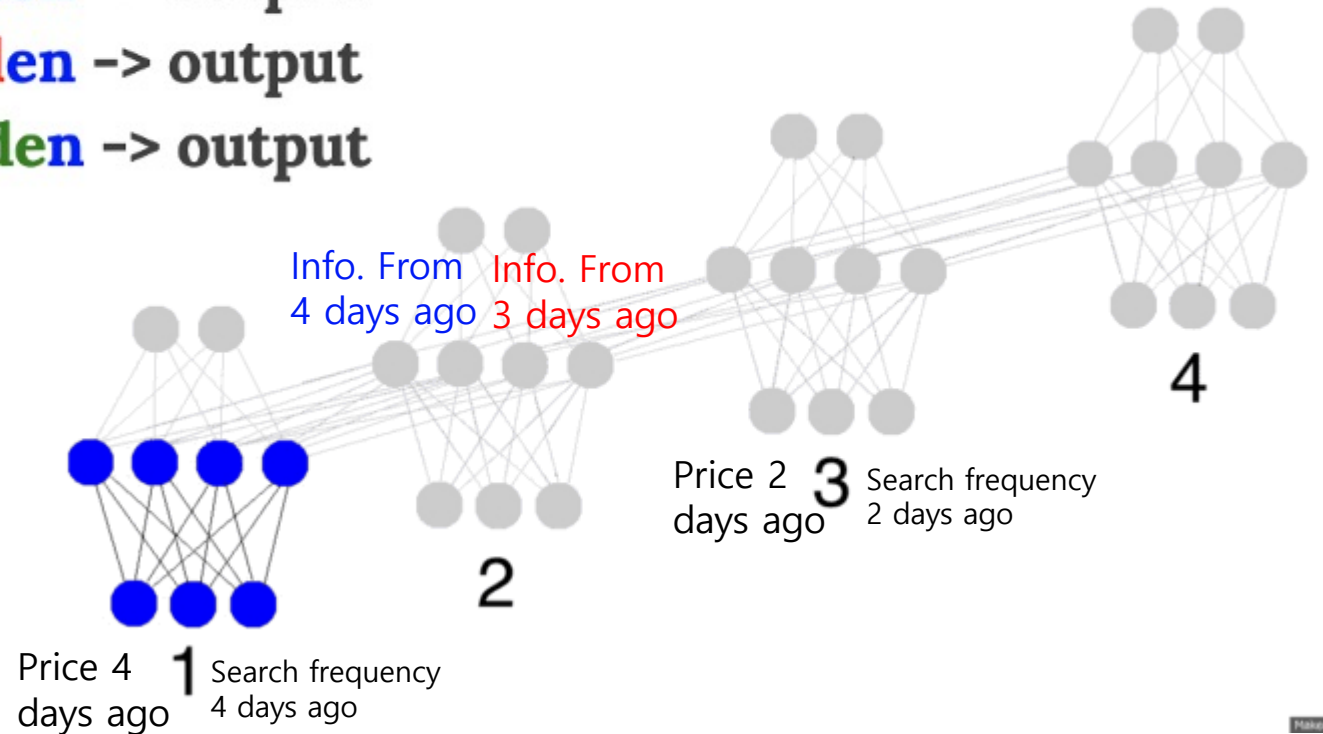
Let's illustrate the transmitted information with colors for each time step.

(**input** + empty_hidden) -> **hidden** -> **output**

(**input** + prev_hidden) -> **hidden** -> **output**

(**input** + prev_hidden) -> **hidden** -> **output**

(**input** + prev_hidden) -> **hidden** -> **output**



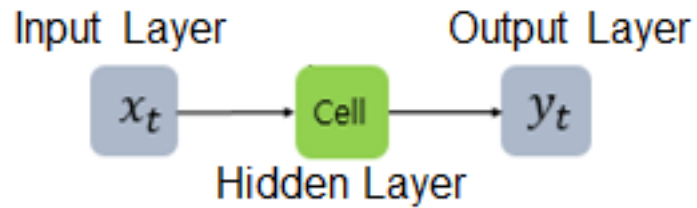
CONTENTS

—

- ① Introduction of RNN
- ② Structure of RNN
- ③ Application of RNN

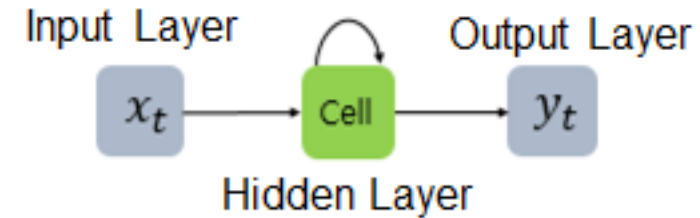
Structure of RNN

📌 Comparison of FFN and RNN



FFN

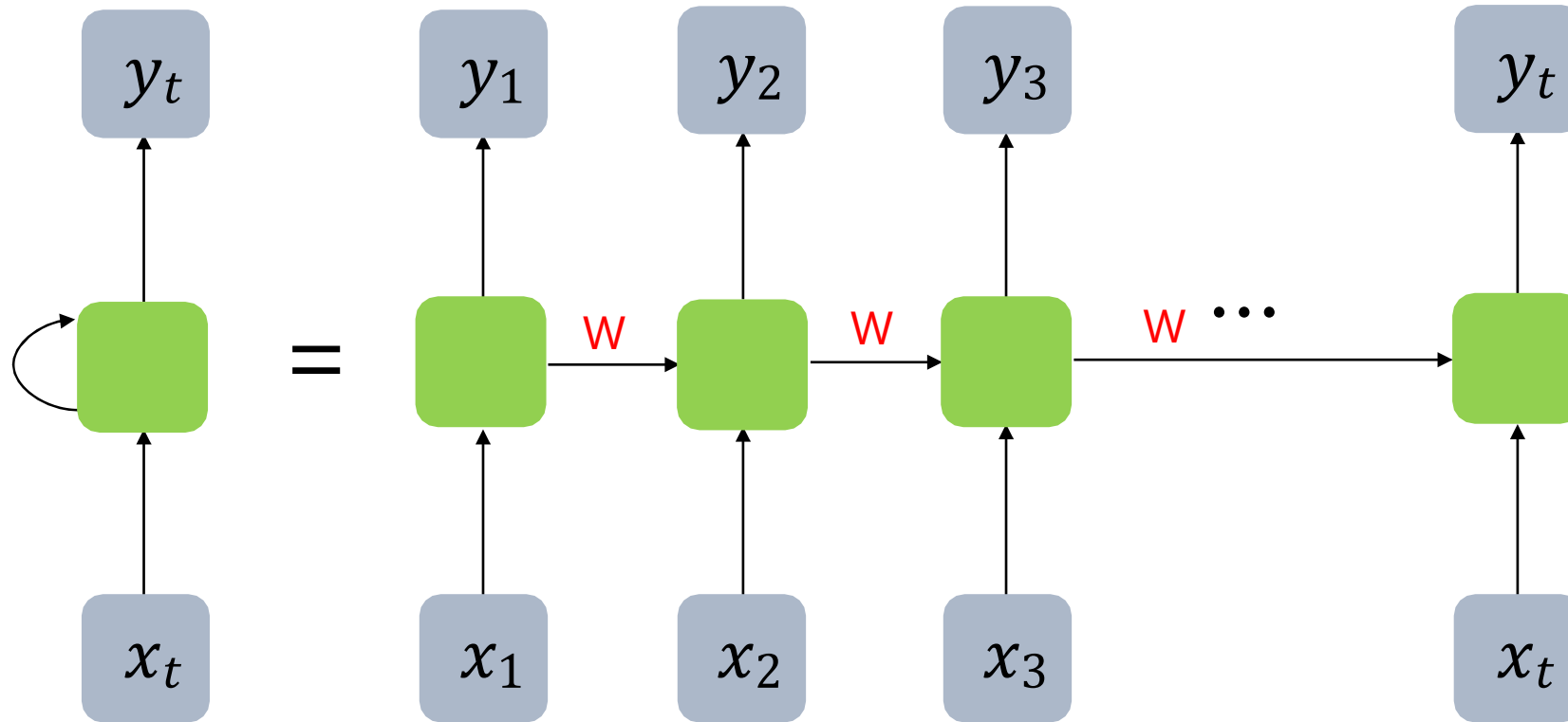
Core idea:
Apply the same weights **W repeatedly!**



RNN

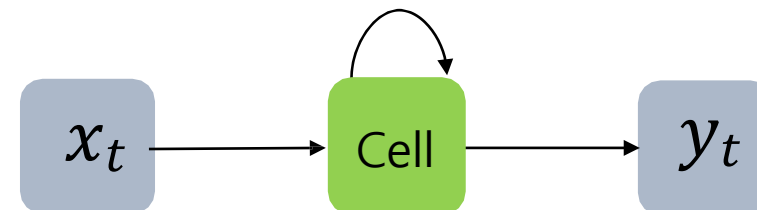
Structure of RNN

Visualization of RNN



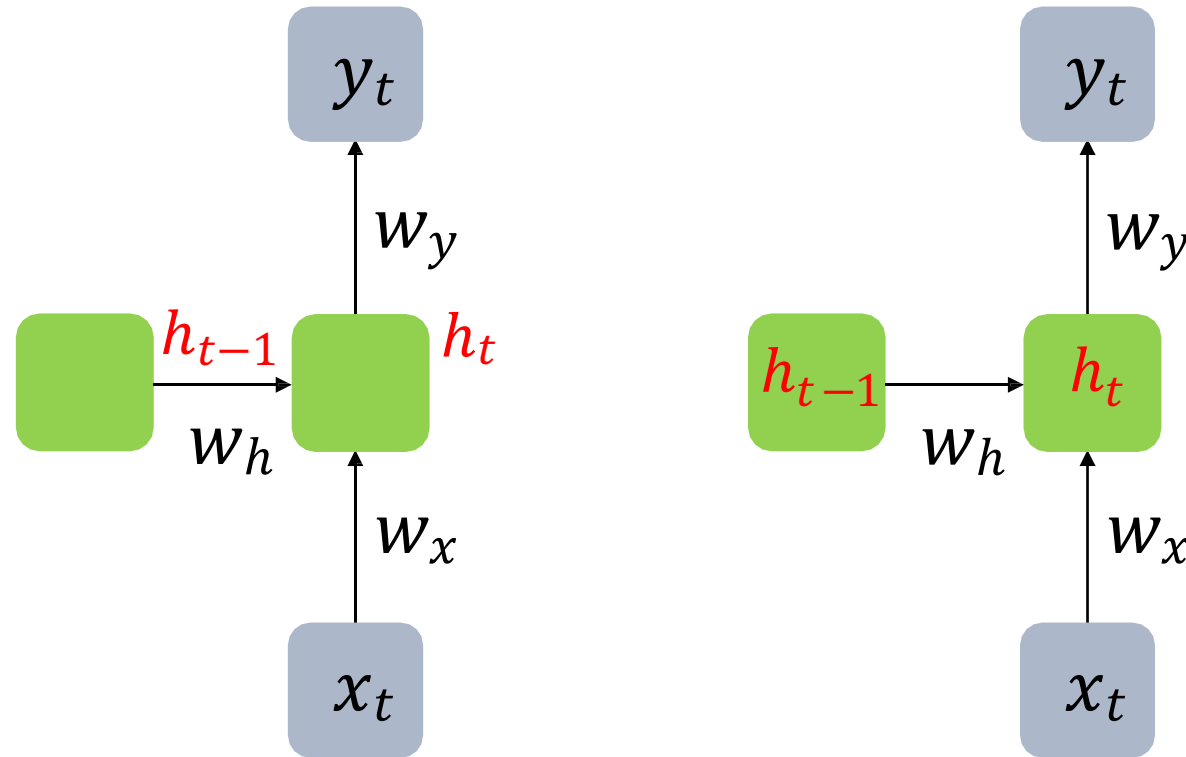
Core idea:

- Apply the same weights W repeatedly!
- The RNN illustrated below fundamentally assumes input and output as vectors.



Structure of RNN

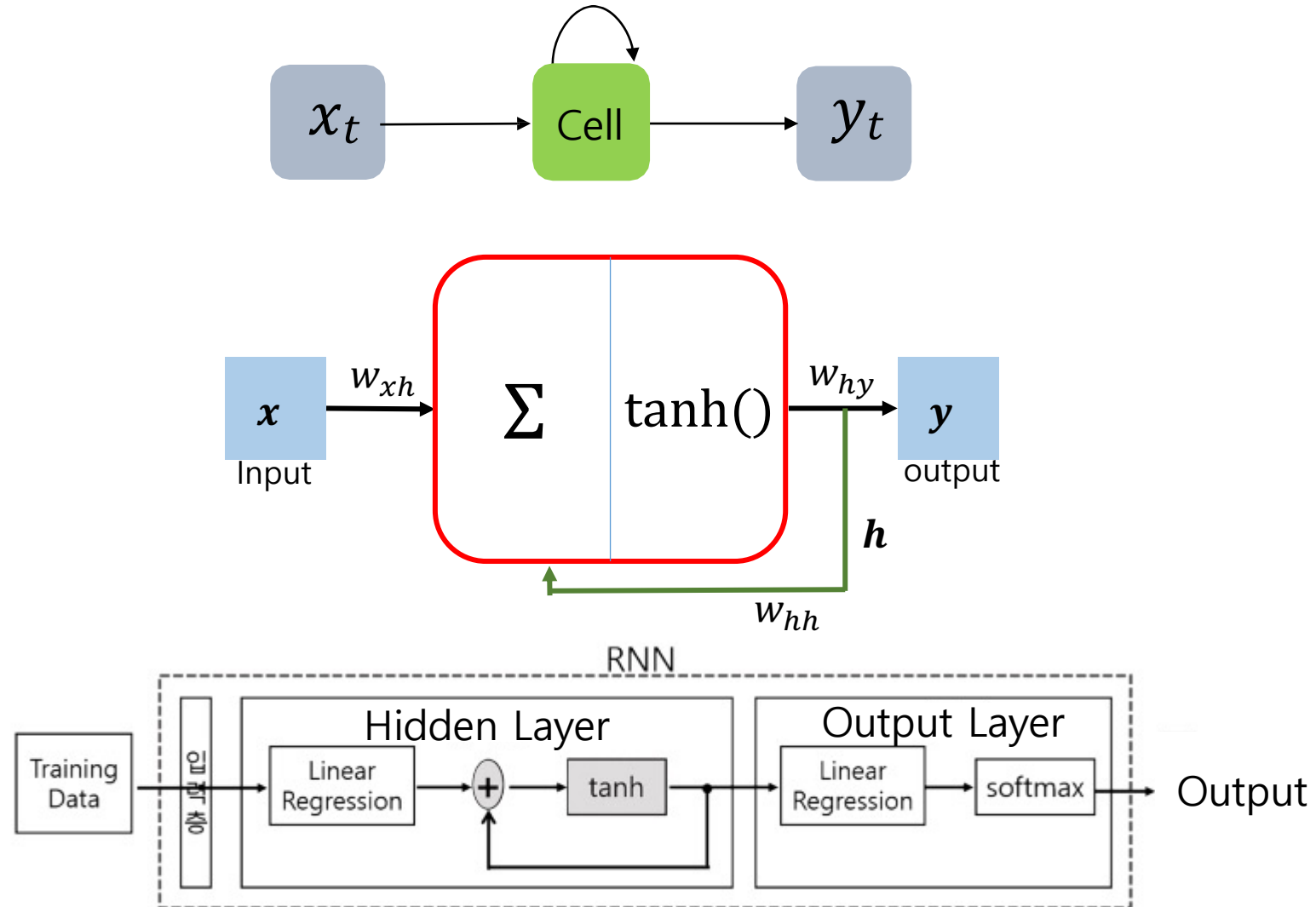
Visualization of RNN



Core idea:
Apply the same weights **W repeatedly!**

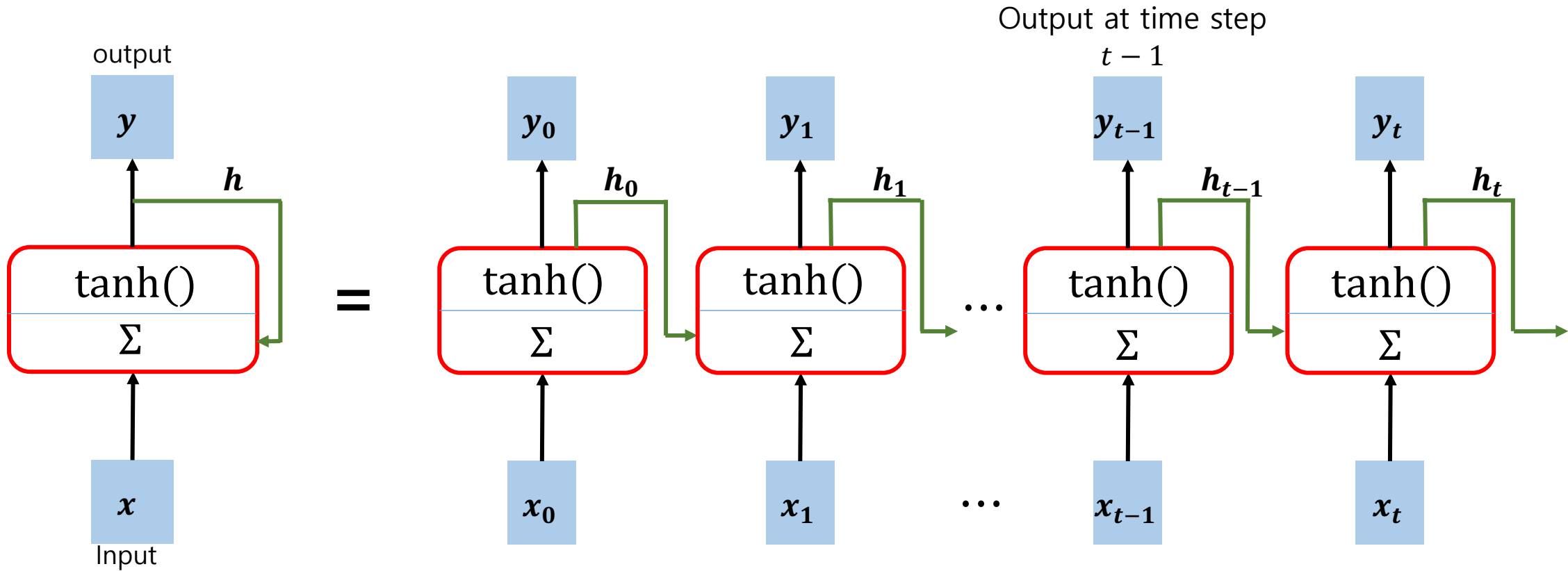
Structure of RNN

Visualization of RNN



Structure of RNN

Visualization of RNN



x_t : Input values for all samples

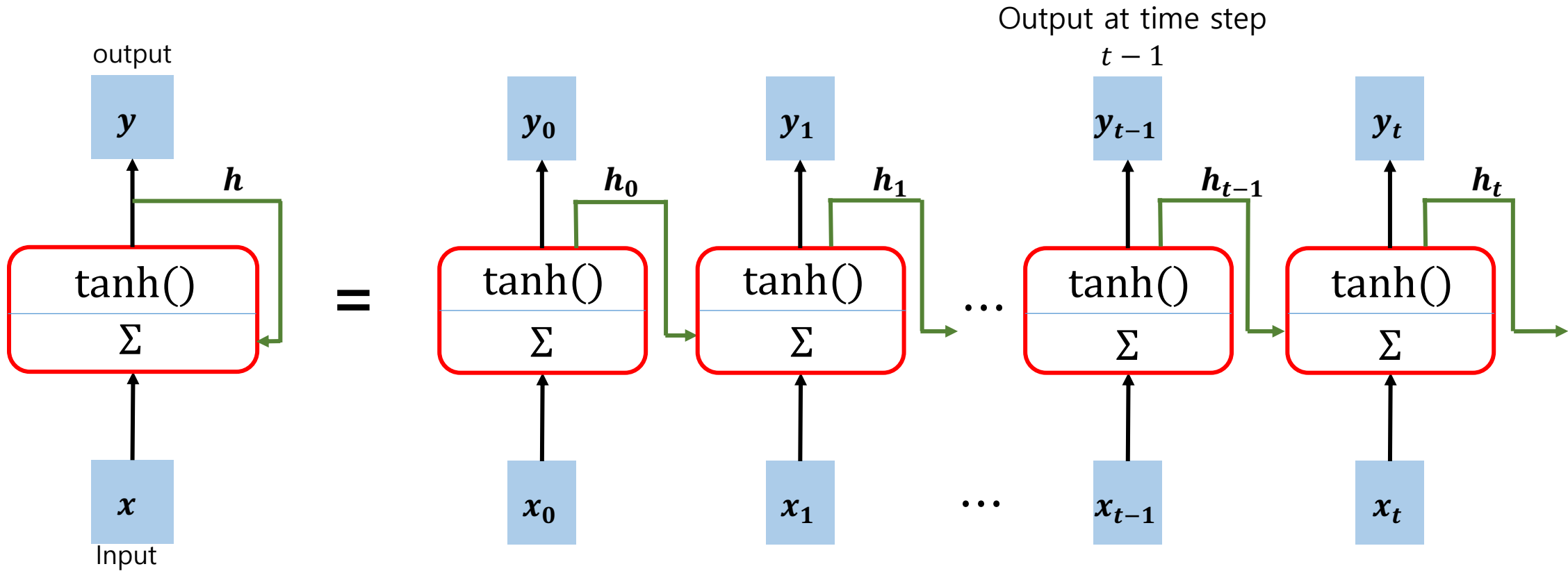
y_t : Output of the recurrent layer for each sample at time step t

h_t : hidden state, defined as $h_t = f(h_{t-1}, x_t)$, meaning the current hidden state is determined by the previous hidden state and the current input.

Hidden state : Information to be passed on to the next time step

Structure of RNN

Visualization of RNN



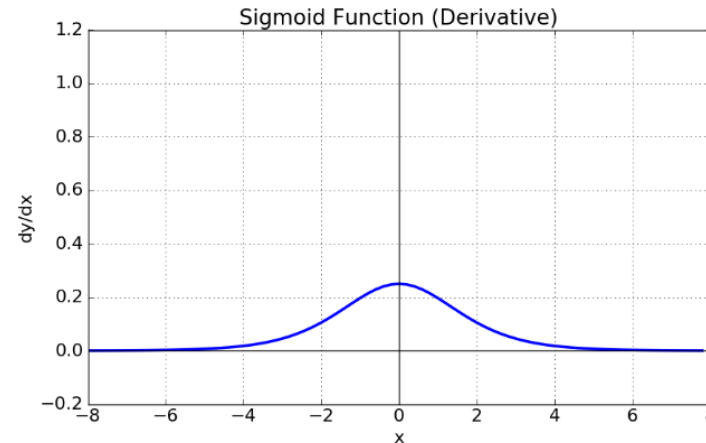
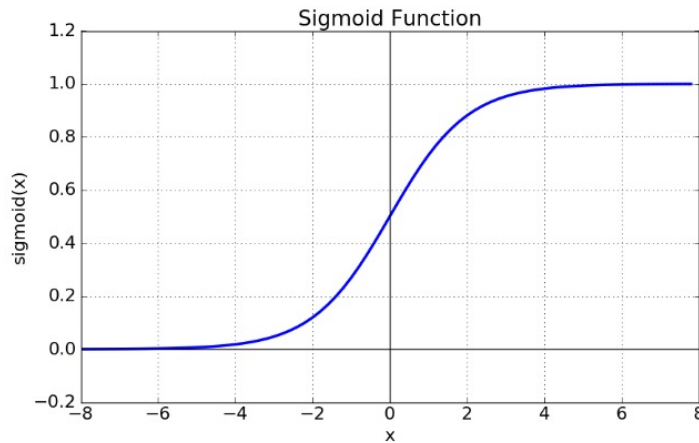
$$h_t = \tanh(X_t \cdot W_{xh} + h_{t-1} \cdot W_{hh} + b)$$

$$y_t = W_{hy} \cdot h_t$$

Structure of RNN

✎ Why Hyperbolic Tangent?

- The sigmoid function outputs values close to 0 for negative inputs. Its derivative reaches a maximum of 0.25, leading to the problem of Vanishing Gradient.

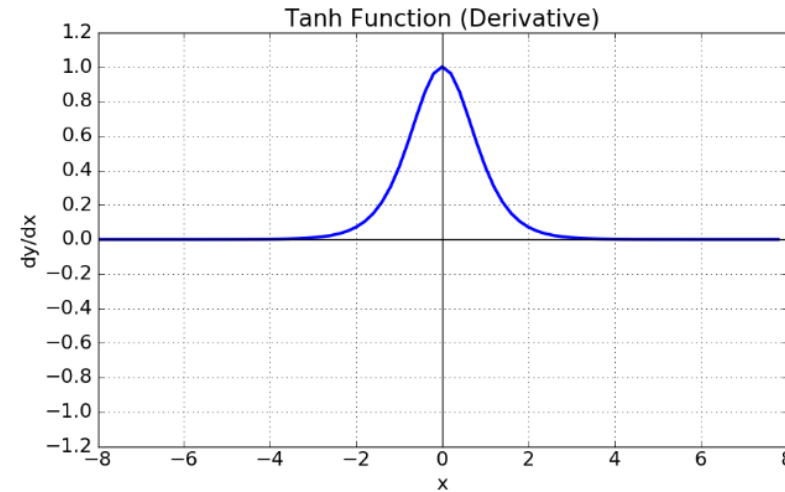
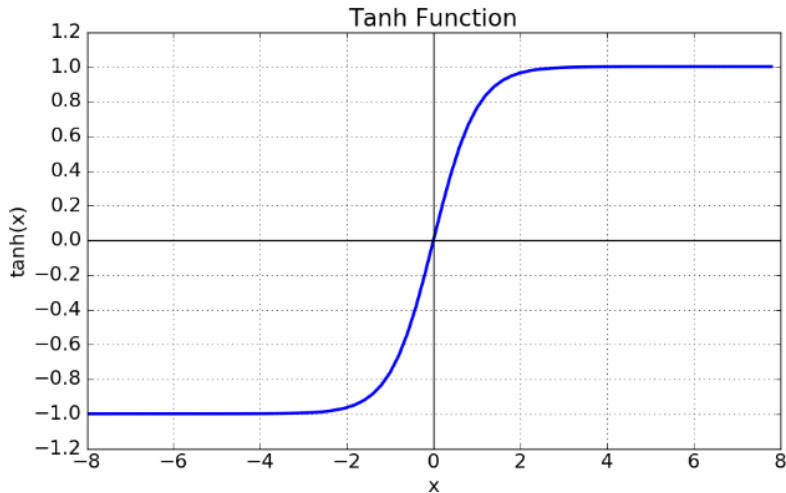


$$\frac{d}{dx} \text{sigmoid}(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x))$$

Structure of RNN

✎ Why Hyperbolic Tangent?

- To mitigate the Vanishing Gradient issue in RNNs, the Tanh function is used
 - The derivative of Tanh function has a maximum value of 1



$$h_t = \tanh(X_t \cdot W_{xh} + h_{t-1} \cdot W_{hh} + b)$$

$$y_t = W_{hy} \cdot h_t$$

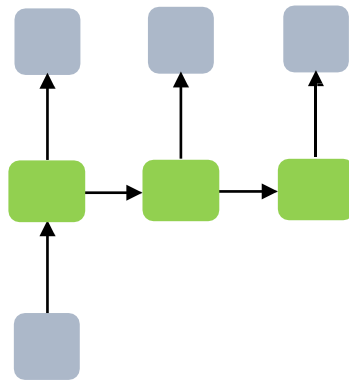
$$\tanh(x) = 2\sigma(2x) - 1$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

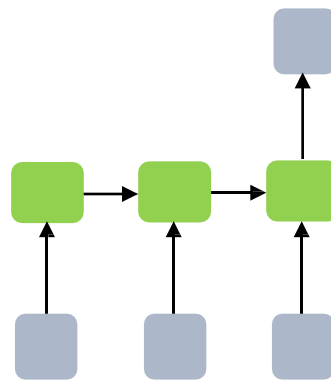
$$\tanh'(x) = 1 - \tanh^2(x)$$

Structure of RNN

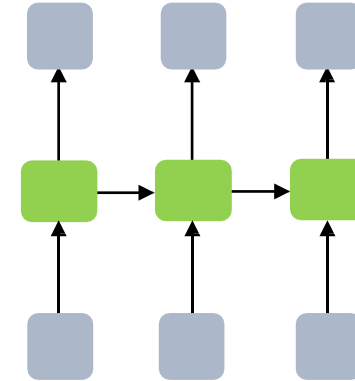
✎ Structure of RNN



(one-to-many)



(many-to-one)

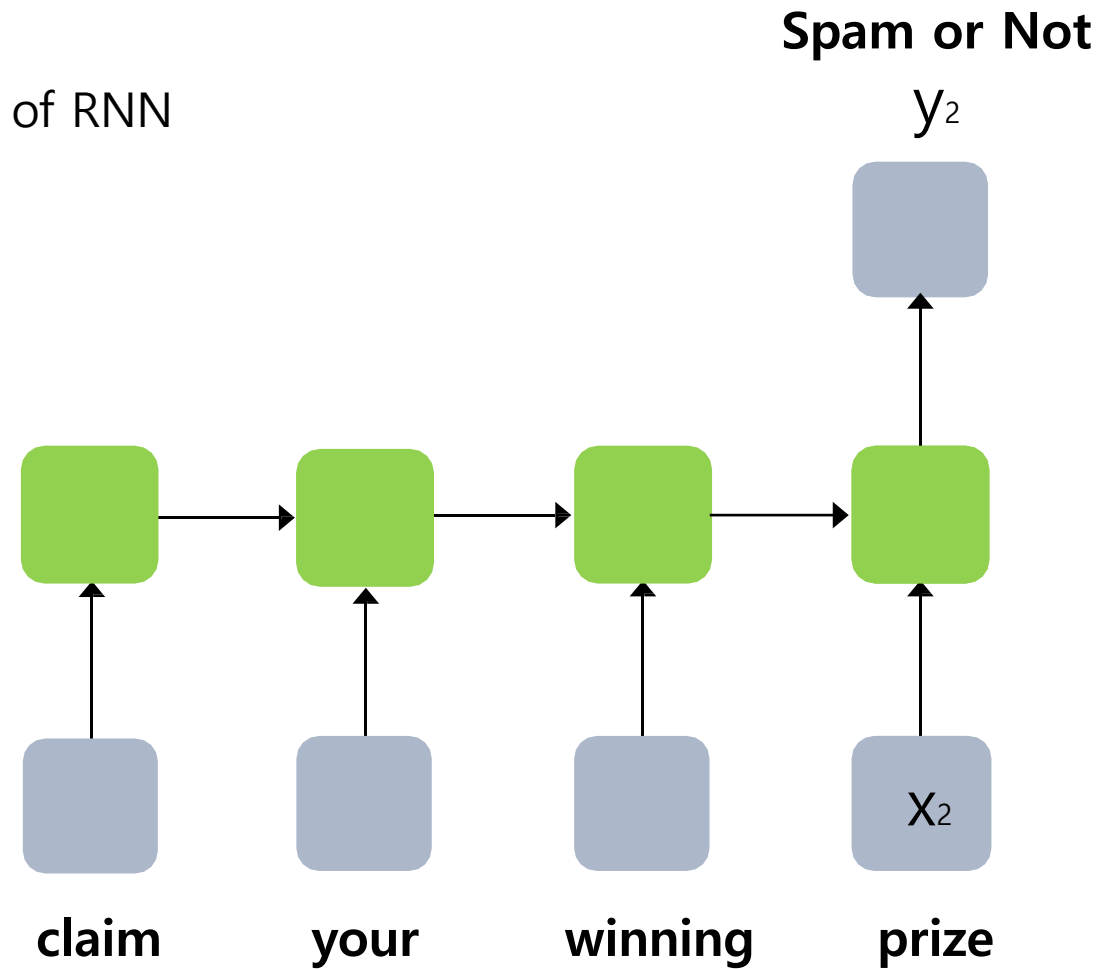


(many-to-many)

Structure of RNN

Structure of RNN

many-to-one Structure of RNN

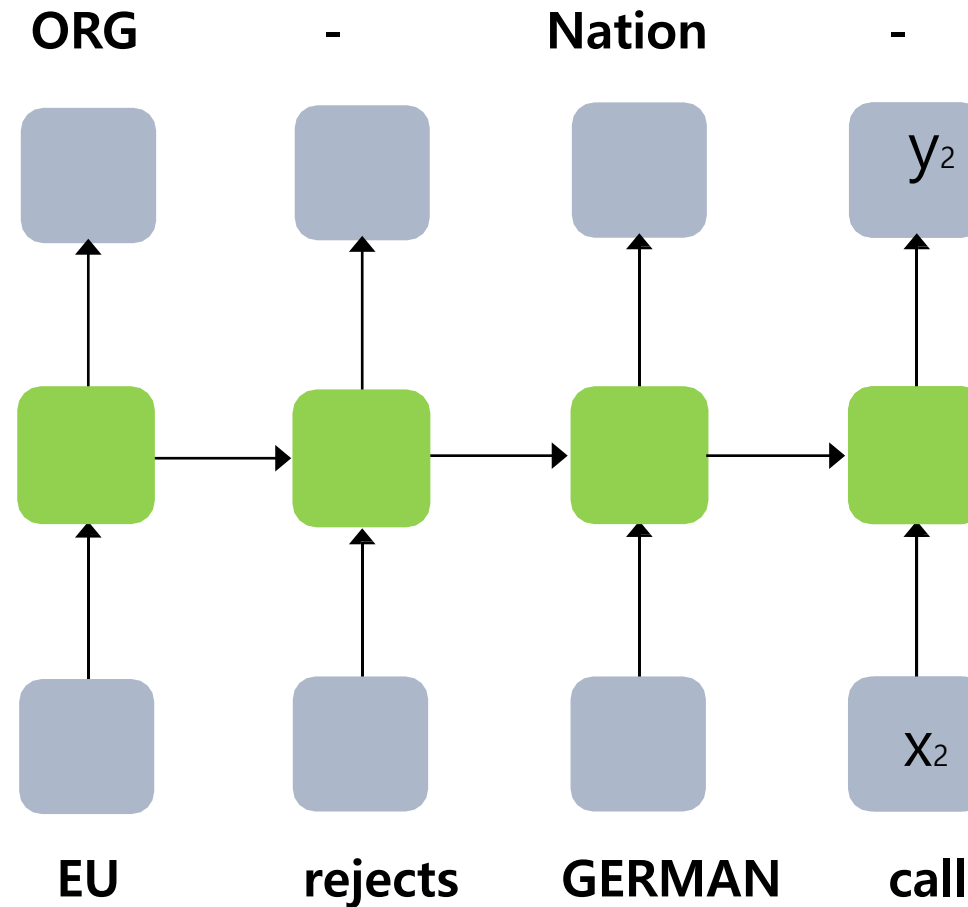


Spam Mail Recognizer

Structure of RNN

Structure of RNN

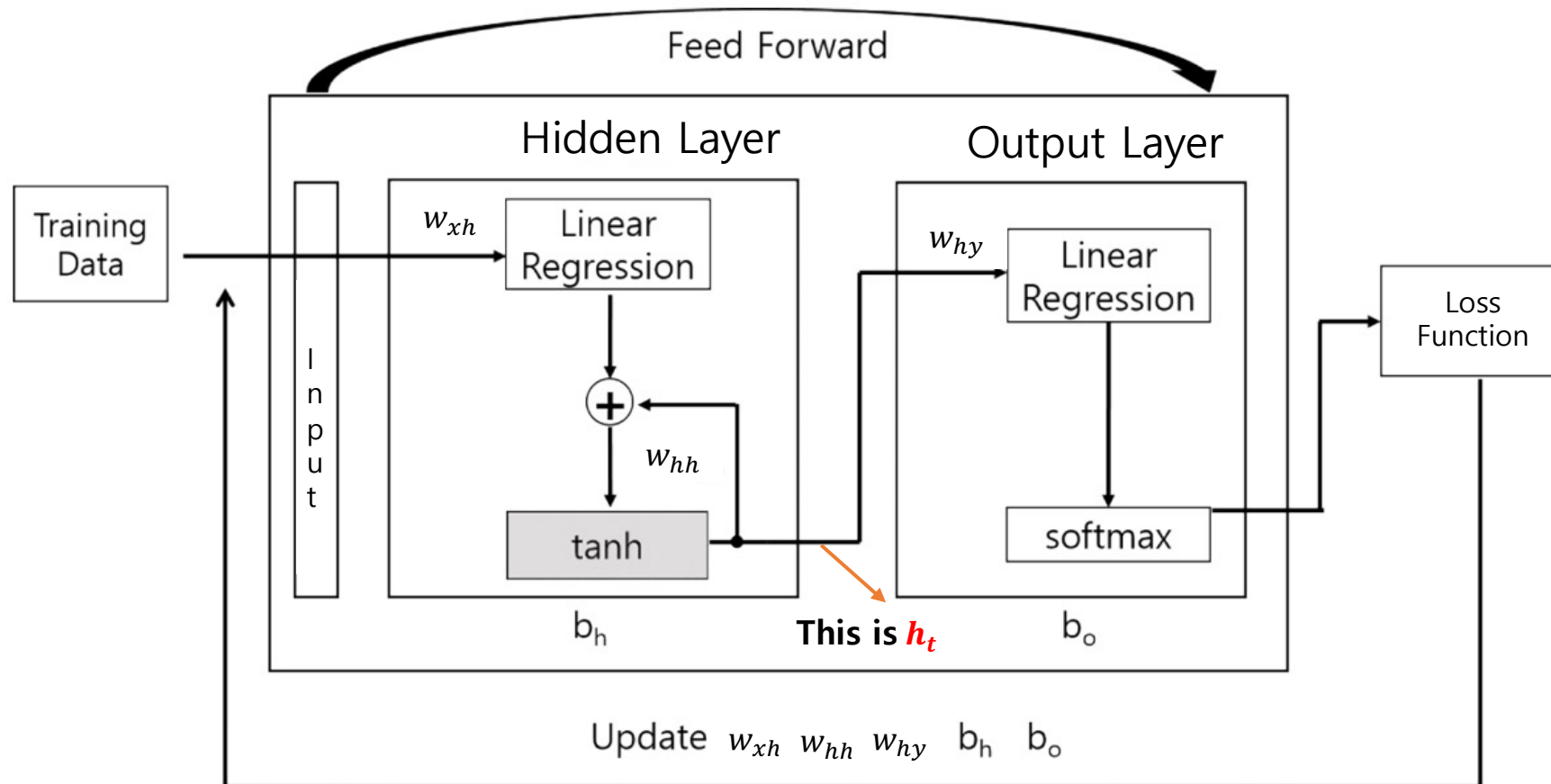
many-to-many Structure of RNN



Named Entity Recognition

RNN Training

Training Structure of RNN



$$h_t = \tanh(X_t \cdot W_{xh} + h_{t-1} \cdot W_{hh} + b)$$

$$y_t = W_{hy} \cdot h_t$$

RNN Training

✎ Training Structure of RNN

Representation of RNN with vector and matrix operations

The diagram illustrates the hidden state update in an RNN using matrix and vector operations. It shows the following components and their dimensions:

- W_h : A green 4x4 matrix with dimension $D_h \times D_h$.
- h_{t-1} : A red 4x1 vector with dimension $D_h \times 1$.
- W_x : A green 4x4 matrix with dimension $D_h \times d$.
- x_t : A blue 4x1 vector with dimension $d \times 1$.
- b : A grey 4x1 vector with dimension $D_h \times 1$.
- h_t : A red 4x1 vector with dimension $D_h \times 1$.

The equation is represented as:

$$\tanh \left(W_h \times h_{t-1} + W_x \times x_t + b \right) = h_t$$

d : Dimension of the word at time-step t
 D_h : Size of the hidden layer

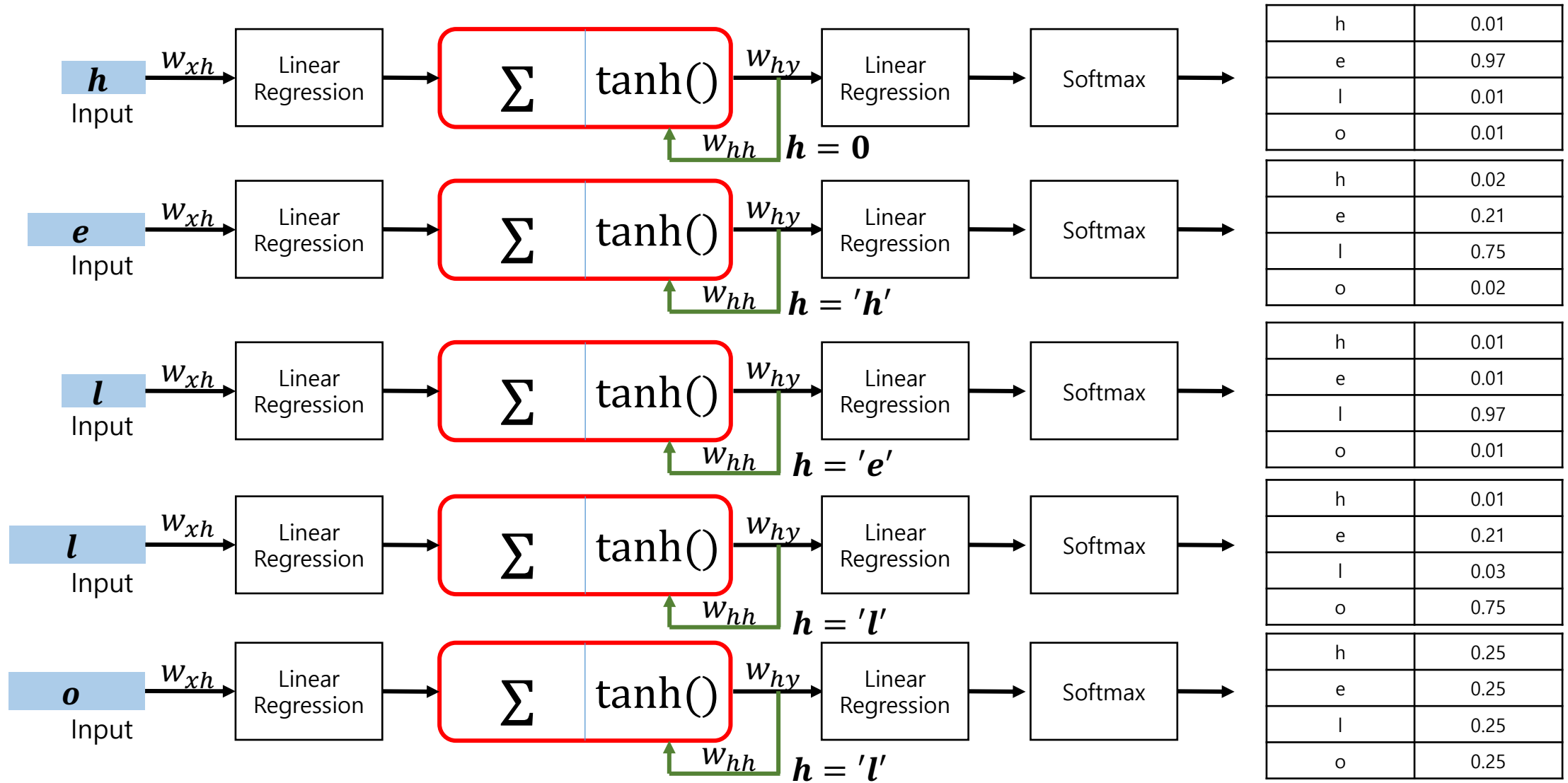
CONTENTS

- ① Introduction of RNN
- ② Structure of RNN
- ③ Application of RNN

RNN Operation

Next character prediction model

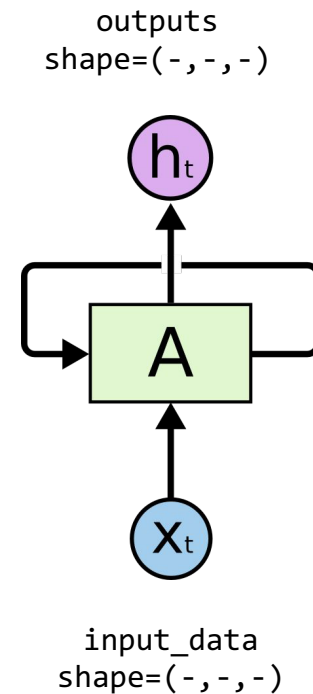
When 'h' is input, predict 'e'
When 'e' is input, predict 'l'



Implementation of RNN (With RNN class)

🖋 Basic implementation form of RNN

```
rnn = torch.nn.RNN(input_size, hidden_size)
outputs, _status = rnn(input_data)
```



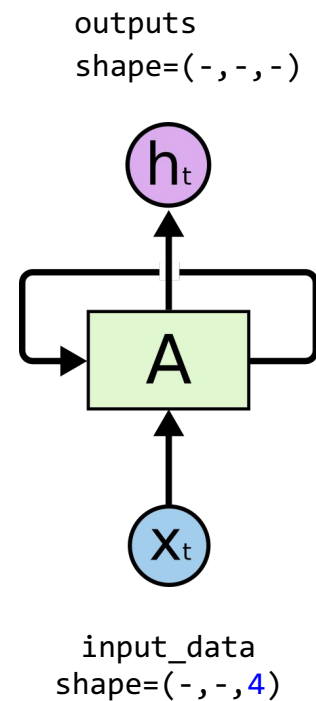
Implementation of RNN (With RNN class)

🖋 RNN input size and format (one-hot)

“hello”

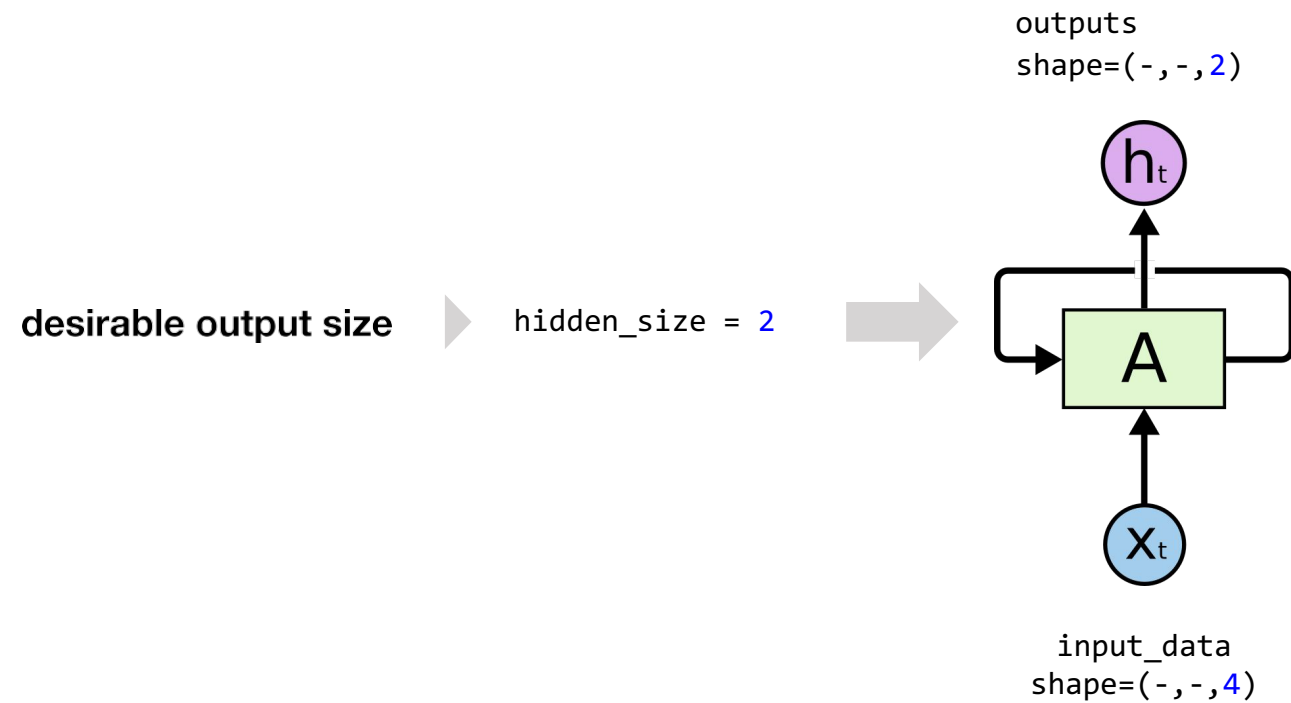
```
# 1-hot encoding  
h = [1, 0, 0, 0]  
e = [0, 1, 0, 0]  
l = [0, 0, 1, 0]  
o = [0, 0, 0, 1]
```

input_size = 4



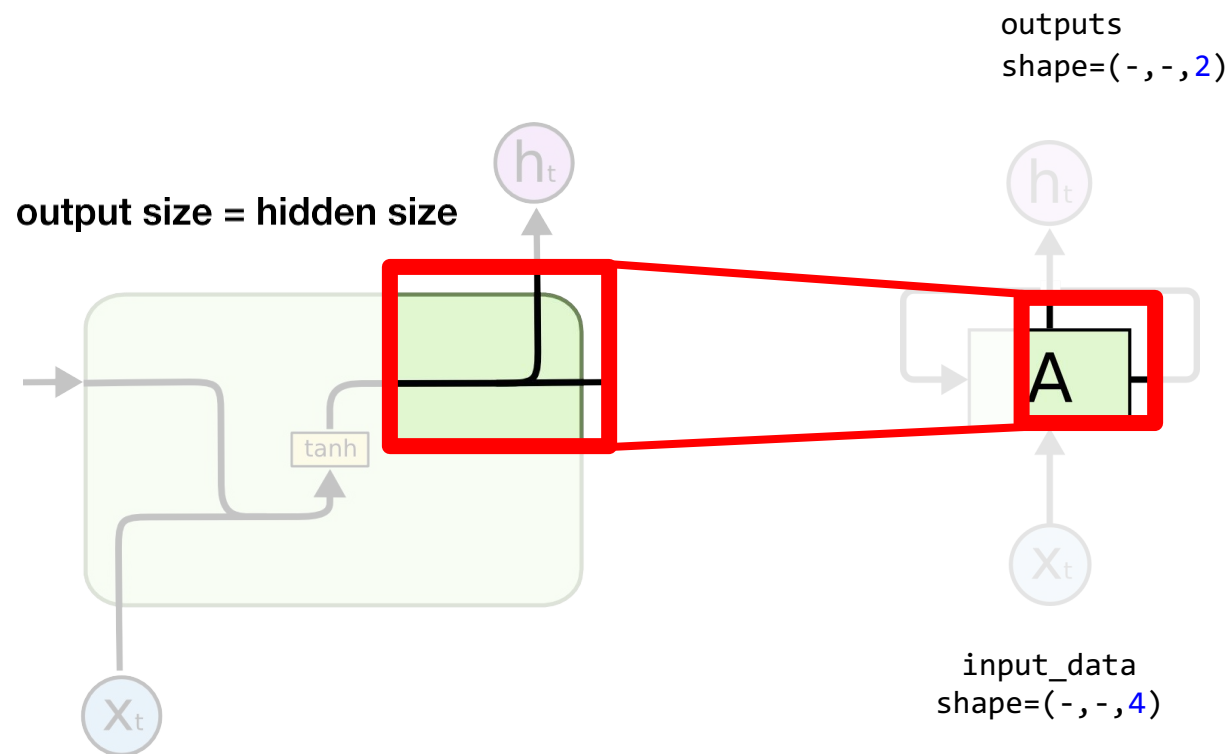
Implementation of RNN (With RNN class)

🖋 RNN's hidden state



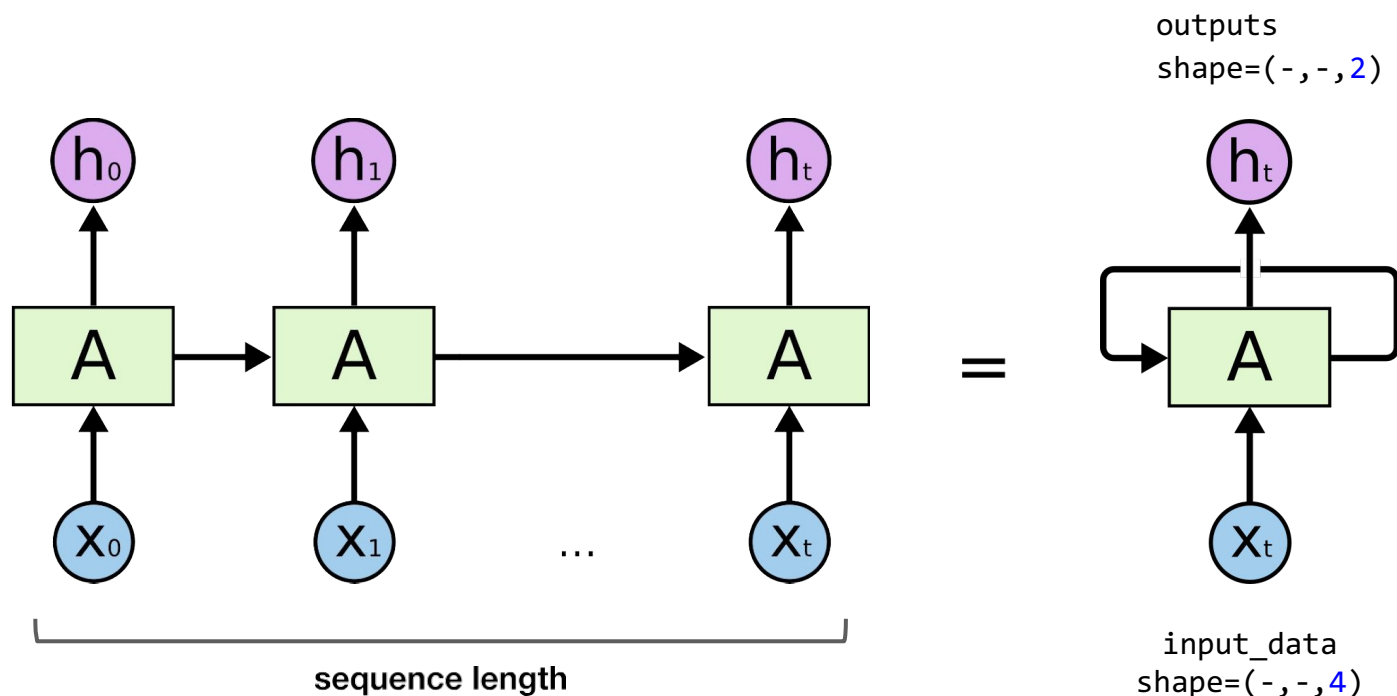
Implementation of RNN (With RNN class)

🖋 RNN's output size



Implementation of RNN (With RNN class)

🖋 Sequence length of RNN input



Implementation of RNN (With RNN class)

📌 Sequence length of RNN input

h, e, l, l, o

$$x_0 = [1, 0, 0, 0]$$

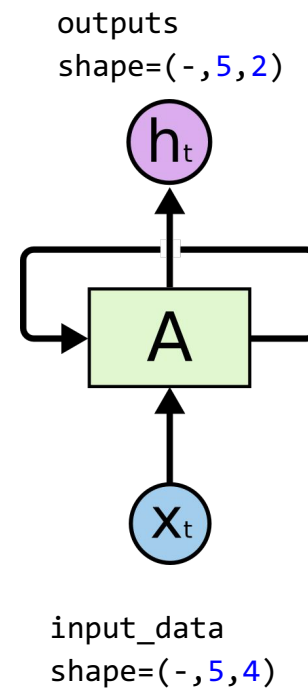
$$x_1 = [0, 1, 0, 0]$$

$$x_2 = [0, 0, 1, 0]$$

$$x_3 = [0, 0, 1, 0]$$

$$x_4 = [0, 0, 0, 1]$$

Automatically Calculated



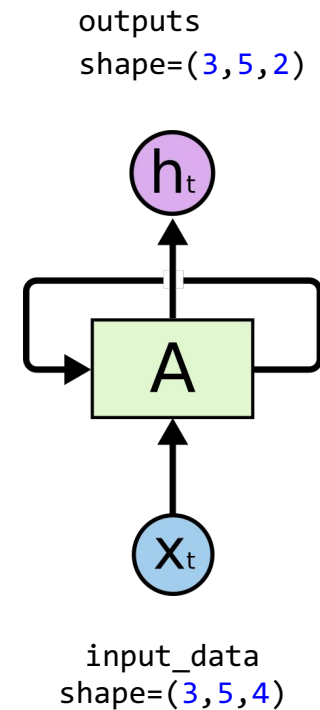
Implementation of RNN (With RNN class)

📝 Batch size for processing multiple samples simultaneously

$$\begin{bmatrix} \text{h, e, l, l, o} \\ \text{e, o, l, l, l} \\ \text{l, l, e, e, l} \end{bmatrix}$$

Set
batch input

Automatically Calculated



Implementation of RNN (With RNN class)

Example of Character Sequence Prediction

```
import torch
import numpy as np

input_size = 4
hidden_size = 2

# 1-hot encoding

h = [1, 0, 0, 0]
e = [0, 1, 0, 0]
l = [0, 0, 1, 0]
o = [0, 0, 0, 1]

input_data_np = np.array([[h, e, l, l, o],
                           [e, o, l, l, l],
                           [l, l, e, e, l]],
                           dtype=np.float32)

# transform as torch tensor
input_data = torch.Tensor(input_data_np)

rnn = torch.nn.RNN(input_size, hidden_size)

outputs, _status = rnn(input_data)
```


'Hihello' example

- 'Hihello' problem
- Data setting
 - One hot encoding
- Cross entropy loss
- Code run through

'hihello' problem

- 'h', 'i', 'h', 'e', 'l', 'l', 'o'
- We will predict the next character!
- How can we represent characters in PyTorch?

How can we represent characters?

- We can represent them by index
 - 'h' -> 0
 - 'i' -> 1
 - 'e' -> 2
 - 'l' -> 3
 - 'o' -> 4

```
# list of available characters  
char_set = ['h', 'i', 'e', 'l', 'o']
```

Implementation of RNN (With RNN class)

One-hot encoding

- We need to encode using one-hot encoding!

list of available characters

```
char_set = ['h', 'i', 'e', 'l', 'o']
```

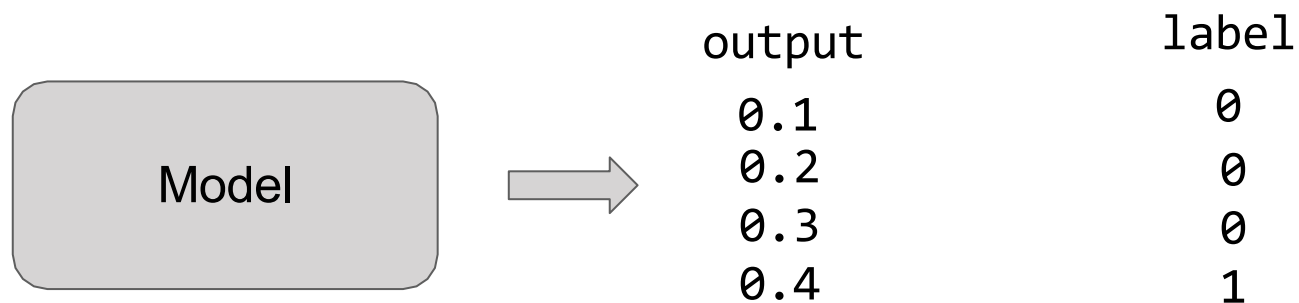
```
x_data = [[0, 1, 0, 2, 3, 3]]
```

```
x_one_hot = [[[1, 0, 0, 0, 0],  
               [0, 1, 0, 0, 0],  
               [1, 0, 0, 0, 0],  
               [0, 0, 1, 0, 0],  
               [0, 0, 0, 1, 0],  
               [0, 0, 0, 1, 0]]]
```

```
y_data = [[1, 0, 2, 3, 3, 4]]
```

Cross Entropy Loss

- Loss for categorical output (usually interpreted as probability)



```
# loss & optimizer setting
criterion = torch.nn.CrossEntropyLoss()
...
loss = criterion(outputs.view(-1, input_size), Y.view(-1))
```

Implementation of RNN (With RNN class)

Code run through (hihello)

```
char_set = ['h', 'i', 'e', 'l', 'o']
# hyper parameters
input_size = len(char_set)
hidden_size = len(char_set)
learning_rate = 0.1
# data setting
x_data = [[0, 1, 0, 2, 3, 3]]
x_one_hot = [[[1, 0, 0, 0, 0],
               [0, 1, 0, 0, 0],
               [1, 0, 0, 0, 0],
               [0, 0, 1, 0, 0],
               [0, 0, 0, 1, 0],
               [0, 0, 0, 1, 0]]]
y_data = [[1, 0, 2, 3, 3, 4]]
```

```
# transform as torch tensor variable
X = torch.FloatTensor(x_one_hot)
Y = torch.LongTensor(y_data)
```

Implementation of RNN (With RNN class)

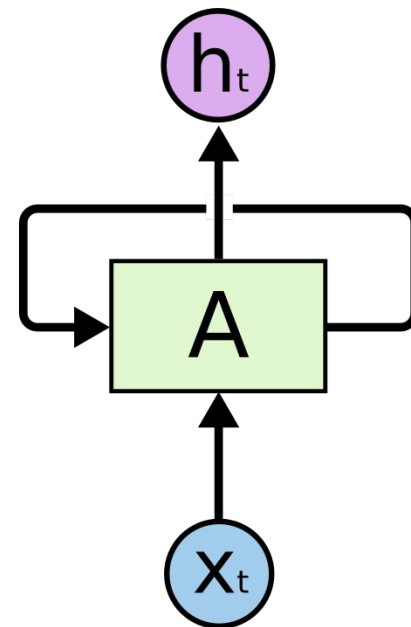
Code run through

```
# declare RNN

rnn = torch.nn.RNN(input_size, hidden_size, batch_first=True) # batch_first guarantees the order of output = (B, S, F)

# loss & optimizer setting
criterion = torch.nn.CrossEntropyLoss()
optimizer = optim.Adam(rnn.parameters(), learning_rate)

# start training
for i in range(100):
    optimizer.zero_grad()
    outputs, _status = rnn(X)
    loss = criterion(outputs.view(-1, input_size), Y.view(-1))
    loss.backward()
    optimizer.step()
    result = outputs.data.numpy().argmax(axis=2)
    result_str = ''.join([char_set[c] for c in np.squeeze(result)])
    print(i, "loss: ", loss.item(), "prediction: ", result, "true Y: ", y_data, "prediction str: ", result_str)
```



Implementation of RNN (With RNN class)

Code run through (charseq)

```
sample = " if you want you"
# make dictionary
char_set = list(set(sample))
char_dic = {c: i for i, c in enumerate(char_set)}

# hyper parameters
dic_size = len(char_dic)
hidden_size = len(char_dic)
learning_rate = 0.1

# data setting
sample_idx = [char_dic[c] for c in sample]
x_data = [sample_idx[:-1]]
x_one_hot = [np.eye(dic_size)[x] for x in x_data]
y_data = [sample_idx[1:]]

# transform as torch tensor variable
X = torch.FloatTensor(x_one_hot)
Y = torch.LongTensor(y_data)
```


Implementation of RNN (With RNN class)

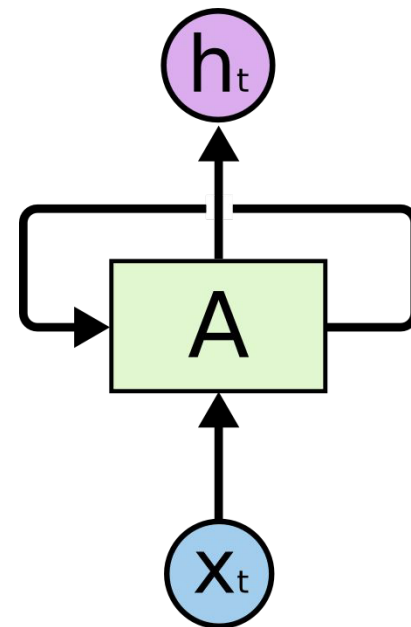
Code run through

```
# declare RNN

rnn = torch.nn.RNN(input_size, hidden_size, batch_first=True)

# loss & optimizer setting
criterion = torch.nn.CrossEntropyLoss()
optimizer = optim.Adam(rnn.parameters(), learning_rate)

# start training
for i in range(100):
    optimizer.zero_grad()
    outputs, _status = rnn(X)
    loss = criterion(outputs.view(-1, input_size), Y.view(-1))
    loss.backward()
    optimizer.step()
    result = outputs.data.numpy().argmax(axis=2)
    result_str = ''.join([char_set[c] for c in np.squeeze(result)])
    print(i, "loss: ", loss.item(), "prediction: ", result, "true Y: ", y_data, "prediction str: ", result_str)
```



Implementation of RNN (With RNN class)

longseq

- We want to use longer dataset
- But we want to train in bigger chunks
- How can we create fixed size sequence dataset from long sentence?

Implementation of RNN (With RNN class)

Making sequence dataset from long sentence

```
sentence = ("if you want to build a ship, don't drum up people together to "  
            "collect wood and don't assign them tasks and work, but rather "  
            "teach them to long for the endless immensity of the sea.")
```

```
"if you wan" -> "f you want"
```

```
"f you want" -> " you want "
```

```
" you want " -> "you want t"
```

```
"you want t" -> "ou want to"
```

```
"ou want to" -> "u want to "
```

...

Implementation of RNN (With RNN class)

Making sequence dataset from long sentence (code)

```
# data setting
x_data = []
y_data = []

for i in range(0, len(sentence) - sequence_length):
    x_str = sentence[i:i + sequence_length]
    y_str = sentence[i + 1: i + sequence_length + 1]
    print(i, x_str, '->', y_str)
    x_data.append([char_dic[c] for c in x_str]) # x str to index
    y_data.append([char_dic[c] for c in y_str]) # y str to index

x_one_hot = [np.eye(dic_size)[x] for x in x_data]

# transform as torch tensor variable
X = torch.FloatTensor(x_one_hot)
Y = torch.LongTensor(y_data)
```

"if you wan" -> "f you want"

"f you want" -> " you want "

" you want " -> "you want t"

"you want t" -> "ou want to"

"ou want to" -> "u want to "

...

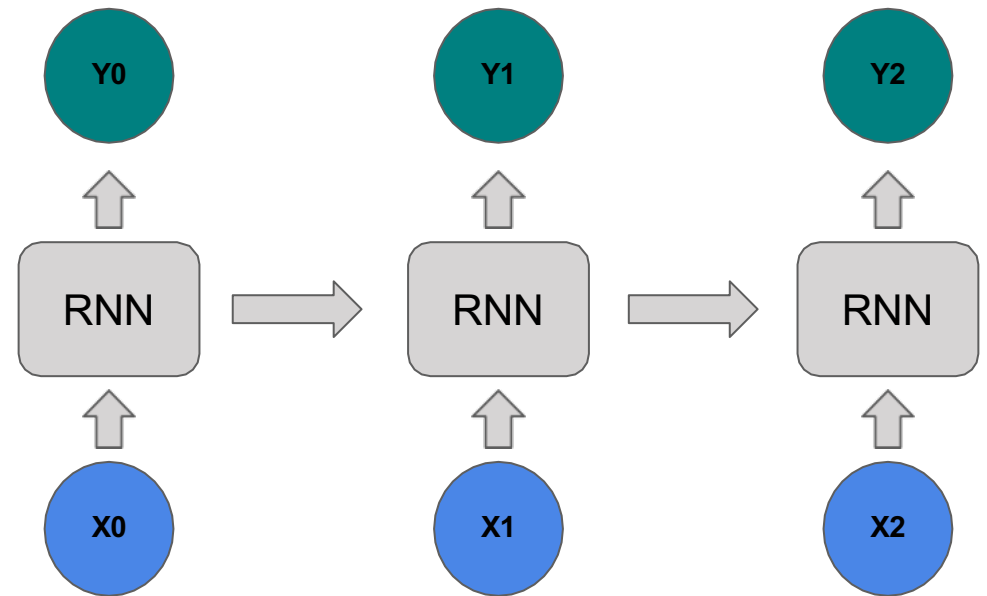
Implementation of RNN (With RNN class)

Adding FC layer and stacking RNN

```
# declare RNN + FC
class Net(torch.nn.Module):
    def __init__(self, input_dim, hidden_dim, layers):
        super(Net, self).__init__()
        self.rnn = torch.nn.RNN(input_dim, hidden_dim, num_layers=layers,
batch_first=True)
        self.fc = torch.nn.Linear(hidden_dim, hidden_dim, bias=True)

    def forward(self, x):
        x, _status = self.rnn(x)
        x = self.fc(x)
        return x
```

```
net = Net(dic_size, hidden_size, 2)
```



Vanilla RNN

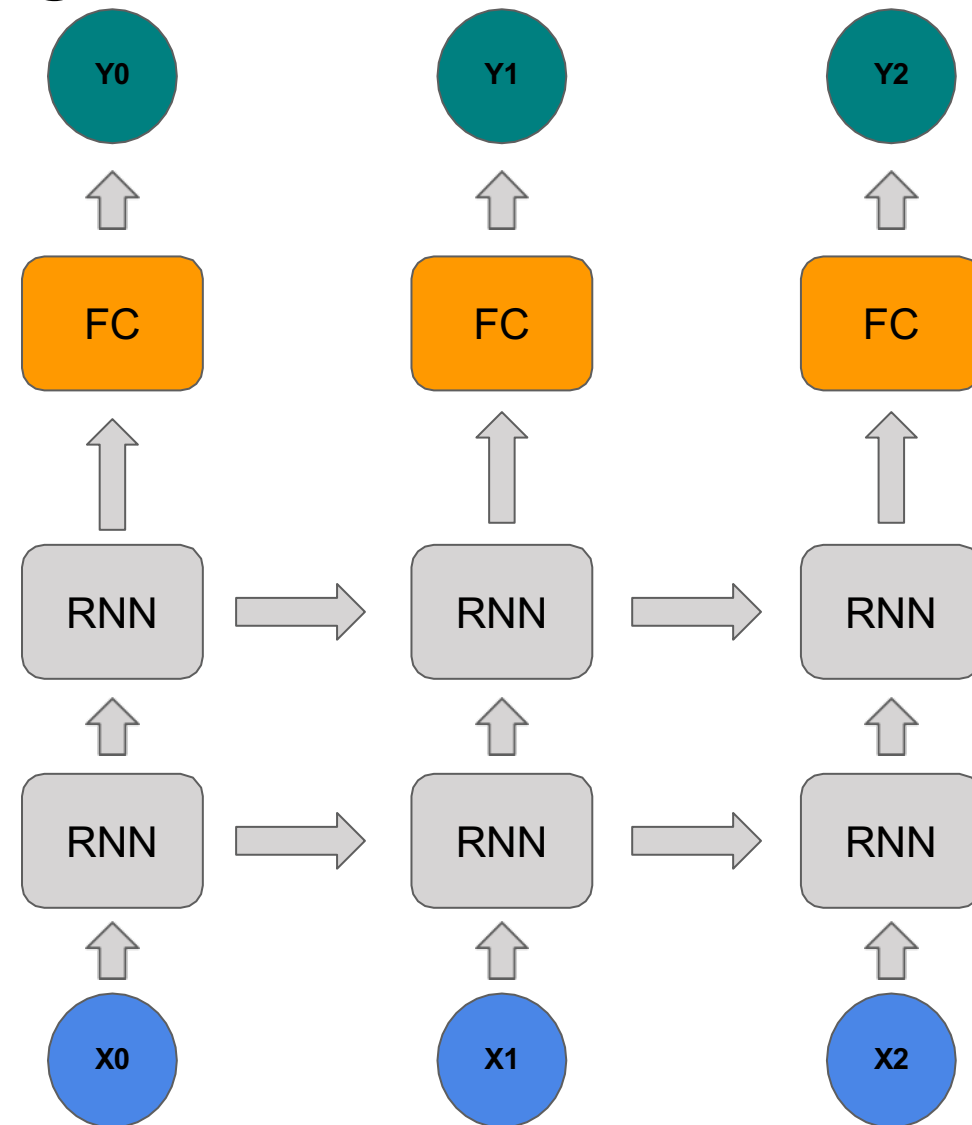
Implementation of RNN (With RNN class)

Adding FC layer and stacking RNN

```
# declare RNN + FC
class Net(torch.nn.Module):
    def __init__(self, input_dim, hidden_dim, layers):
        super(Net, self).__init__()
        self.rnn = torch.nn.RNN(input_dim, hidden_dim, num_layers=layers,
batch_first=True)
        self.fc = torch.nn.Linear(hidden_dim, hidden_dim, bias=True)

    def forward(self, x):
        x, _status = self.rnn(x)
        x = self.fc(x)
        return x

net = Net(dic_size, hidden_size, 2)
```



Implementation of RNN (With RNN class)

Code run through

```
# loss & optimizer setting
criterion = torch.nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), learning_rate)

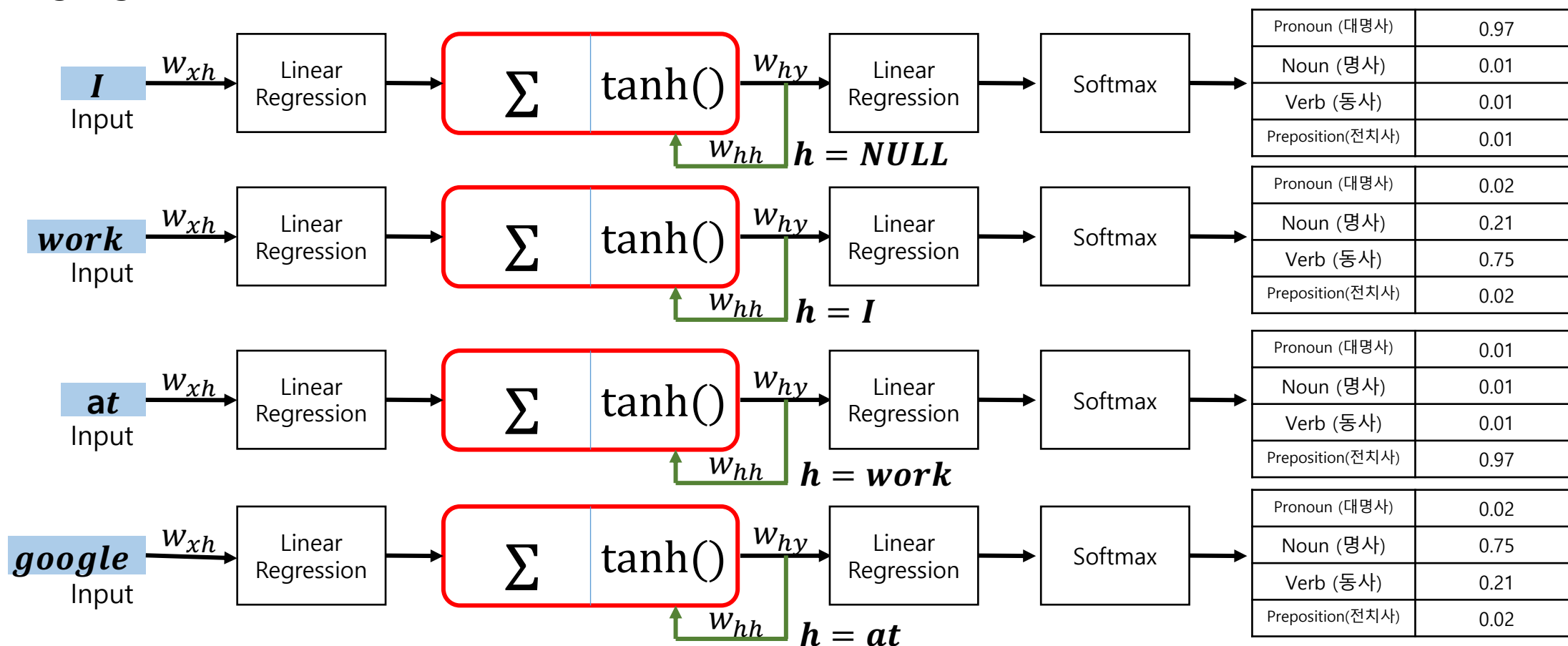
# start training
for i in range(100):
    optimizer.zero_grad()
    outputs = net(X)
    loss = criterion(outputs.view(-1, dic_size), Y.view(-1))
    loss.backward()
    optimizer.step()

results = outputs.argmax(dim=2)
predict_str = ""
for j, result in enumerate(results):
    print(i, j, ''.join([char_set[t] for t in result]), loss.item())
    if j == 0:
        predict_str += ''.join([char_set[t] for t in result])
    else:
        predict_str += char_set[result[-1]]
```

Application of RNN

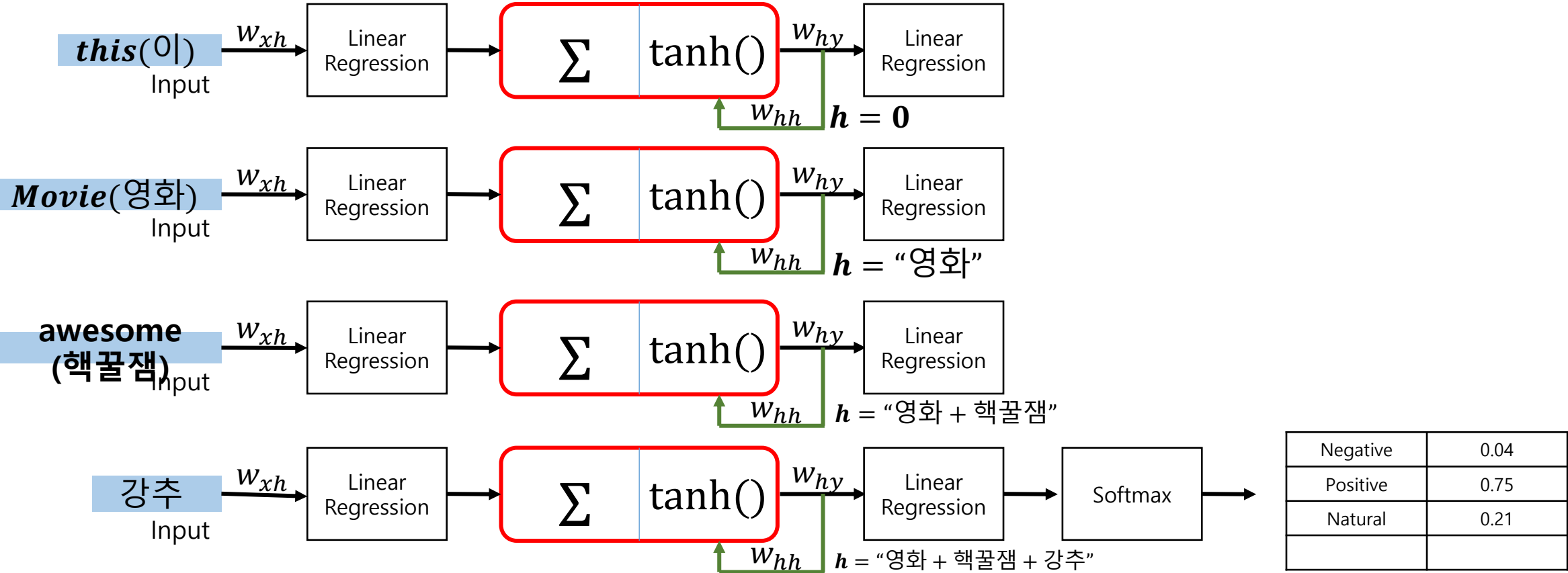
✎ Morphological analyzer

- I work at google → 나는 구글에 근무한다.
- I google at work → 나는 일하면서 구글링한다.



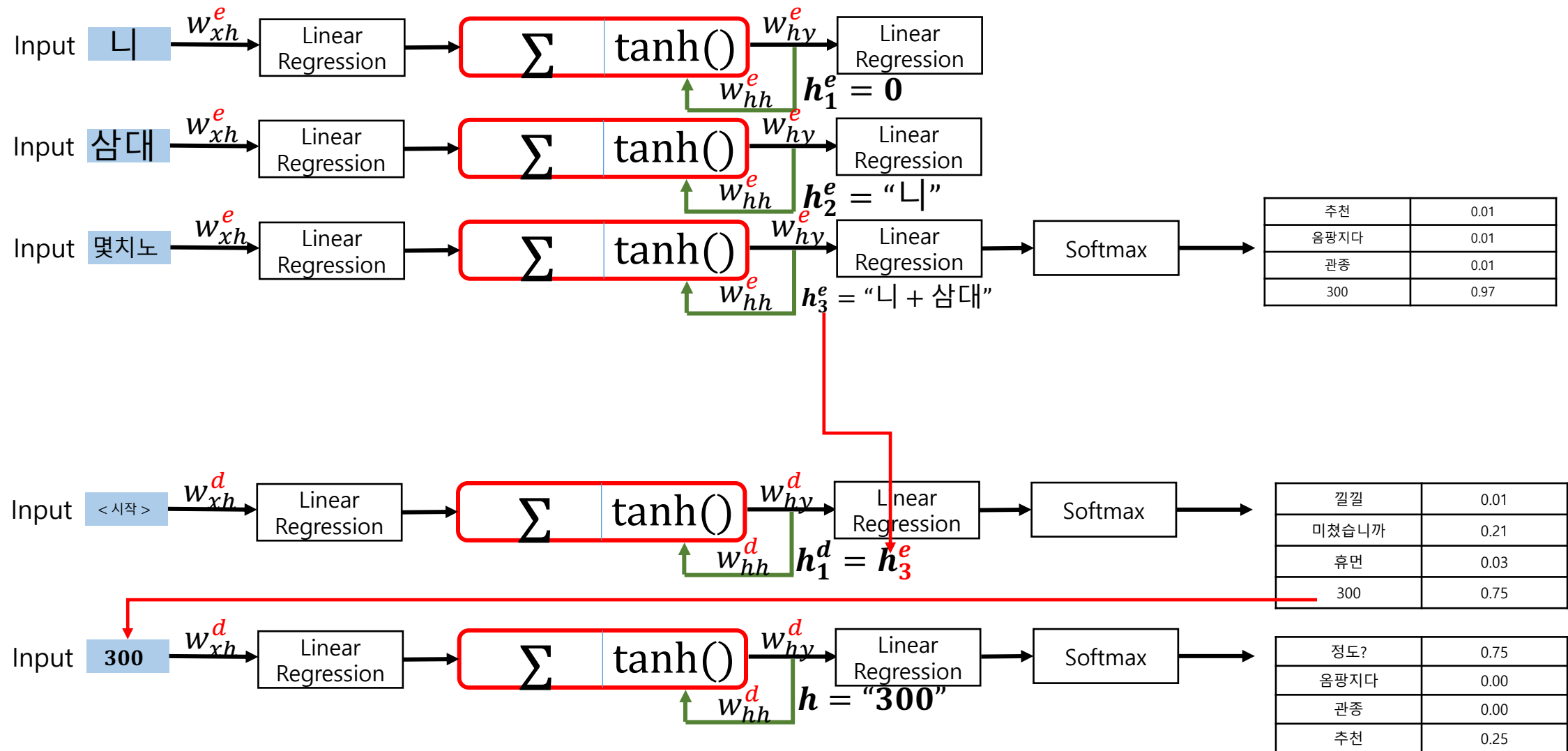
Application of RNN

📝 Sentiment analysis: Is a movie review positive or negative?



Application of RNN

Chatbot (seq2seq will be discussed next week)



Further Discussion

- RNN Training Methods
 - ✓ RTRL (Real-time recurrent learning): Uses stochastic gradient descent for online learning
 - ✓ BPTT (Backpropagation through time): Time-based error backpropagation
- Advantages of RNN
 - Utilizes previous information for solving current problems
- Disadvantages of RNN
 - ✓ Long-term dependency: Difficulty handling context from distant past states due to gradient vanishing
 - Resolved by LSTM (Long Short Term Memory) and GRU (Gated Recurrent Units)



Thank you.